PPKS 10: Digitalne modulacije

U prethodnim predavanjima obrađene su analogne modulacije kod kojih je informacijski signal kontinuiran. To znači da signal može poprimiti beskonačno mnogo različitih vrijednosti u vremenu. Na primjer, govorni signal ili glazba predstavljaju kontinuirane promjene amplitude tijekom vremena. Kod amplitudne modulacije (AM) amplituda nosioca kontinuirano se mijenja u skladu s trenutnom vrijednošću informacijskog signala. Slično tome, kod frekvencijske modulacije (FM) frekvencija nosioca kontinuirano se mijenja, dok se kod fazne modulacije (PM) kontinuirano mijenja faza nosioca.

Kod digitalnih komunikacijskih sustava situacija je drugačija. Informacija više nije zapisana kontinuiranim signalom nego nizom diskretnih vrijednosti, odnosno bitova. **Bit** je najmanja jedinica digitalne informacije i može poprimiti samo dvije vrijednosti: 0 i 1. Dakle, za razliku od analognog informacijskog signala, kod digitalnog koristimo konačan broj unaprijed definiranih stanja. U najjednostavnijem obliku imamo samo dva stanja (jedno stanje koje predstavlja 0 i jedno za 1). Upravo je to osnovna razlika između analognih i digitalnih modulacija.

Bitovi se najčešće ne prenose pojedinačno nego se grupiraju u simbole. **Simbol** koji sadrži n bitova može predstavljati

$$M = 2^n$$

različitih stanja moduliranog signala. Zato se kod digitalnih modulacija često govori o broju simbola ili broju stanja modulacije. Na primjer BPSK ima (M=2) stanja, QPSK (M=4) stanja, a 16-QAM (M=16) mogućih stanja signala. To znači da BPSK ima 1 bit u simbolu, QPSK 2, a 16-QAM 4 bita u simbolu.

Konstelacijski dijagram

U prethodnom poglavlju pokazali smo da se analitički signal može prikazati kao rotirajući vektor u kompleksnoj ravnini. Vrh tog vektora u svakom se trenutku nalazi u jednoj točki čiji položaj određuje trenutnu amplitudu i fazu signala.

Kod digitalnih modulacija informacija se ne prenosi kontinuiranim promjenama signala nego nizom diskretnih simbola. Svakom simbolu pridružena je jedna unaprijed definirana vrijednost amplitude i faze. Tijekom trajanja pojedinog simbola te vrijednosti ostaju nepromijenjene, a mijenjaju se tek pri prijelazu na sljedeći simbol. Zato jedno stanje signala možemo opisati jednom točkom u

kompleksnoj ravnini te ga pridružiti jednom simbolu.

Dakle, signal jednog simbola može se zapisati kao

$$s(t) = I\cos(2\pi f_c t) - Q\sin(2\pi f_c t),$$

gdje je I *in-phase*, a Q *quadrature* komponenta simbola.

Komponenta I određuje koliko je prisutno kosinusnog nosioca, dok komponenta Q određuje koliko je prisutno sinusnog nosioca iste frekvencije. Budući da su sinus i kosinus međusobno fazno pomaknuti za 90°, kažemo da su te dvije komponente ortogonalne. Ideja je slična Fourierovoj analizi. Tamo signal opisujemo pomoću koeficijenata uz sinusne i kosinusne funkcije, a pomoću I i Q komponenti promatramo sinusnu i kosinusnu komponentu na frekvenciji nosioca (f_c).

Vrijednosti I i Q zajedno određuju amplitudu i fazu signala. Umjesto da promatramo dvije odvojene komponente, praktičnije ih je zapisati kao jedan kompleksan broj

$$z = I + jQ.$$

Iz vrijednosti komponenti I i Q amplitudu i fazu možemo odrediti kao

$$A = \sqrt{I^2 + Q^2}$$

$$\varphi = \arctan\left(\frac{Q}{I}\right).$$

Kao i kod analitičkog signala, amplituda odgovara udaljenosti točke od ishodišta kompleksne ravnine, a faza kutu koji vektor zatvara s osi I.

Taj kompleksni broj u potpunosti opisuje stanje simbola pa ga možemo prikazati jednom točkom u kompleksnoj ravnini. Ako sve moguće simbole prikažemo u ravnini čije osi predstavljaju vrijednosti komponenti I i Q, dobivamo konstelacijski dijagram (engl. constellation diagram). Horizontalna os predstavlja in-phase komponentu I (realni dio), dok vertikalna os predstavlja quadrature komponentu Q (imaginarni dio).

Zašto minus?

Kompleksni zapis koristimo isključivo kao matematički alat. Signal koji odašiljemo mora biti realan jer predstavlja veličinu (napon ili struju) koja može imati samo realnu vrijednost u svakom trenutku. Imaginarni dio nema fizikalnog značenja i postoji samo kao posljedica matematičkog formalizma. Zato promatramo što ćemo dobiti za realni dio u produktu kompleksnog

simbola $z = I + jQ$ i kompleksnog nosioca $e^{j2\pi f_c t}$:

$$s(t) = \mathrm{Re}\left\{ z \cdot e^{j2\pi f_c t} \right\}$$

Raspiše li se produkt:

$$z \cdot e^{j2\pi f_c t} = (I + jQ)(\cos 2\pi f_c t + j \sin 2\pi f_c t)$$

$$= I \cos 2\pi f_c t + jI \sin 2\pi f_c t + jQ \cos 2\pi f_c t + j^2 Q \sin 2\pi f_c t$$

Budući da je $j^2 = -1$:

$$= (I \cos 2\pi f_c t - Q \sin 2\pi f_c t) + j(I \sin 2\pi f_c t + Q \cos 2\pi f_c t)$$

Uzimanjem realnog dijela dobivamo signal koji odašiljemo:

$$s(t) = I \cos(2\pi f_c t) - Q \sin(2\pi f_c t)$$

Digitalni pandani analognim modulacijama

Slično kao kod analognih, kod digitalnih se modulacija također mogu mijenjati amplituda, faza i frekvencija nosioca. Svaka analogna modulacija ima svoj digitalni pandan u kojemu se umjesto kontinuiranog informacijskog signala koristi diskretan skup unaprijed definiranih stanja.

Parametar nosioca	Analogna modulacija	Digitalna modulacija
Amplituda	AM	ASK
Frekvencija	FM	FSK
Faza	PM	PSK
Amplituda i faza	—	QAM

Kod diskretne modulacije amplitude (**ASK**, od engl. *Amplitude Shift Keying*) različitim simbolima pridružene su različite amplitude nosioca. Kod diskretne modulacije frekvencije (**FSK**, od engl. *Frequency Shift Keying*) svaki simbol prenosi se na drugoj frekvenciji. Kod diskretne modulacije faze (**PSK**, od engl. *Phase Shift Keying*) amplituda ostaje konstantna, a informacija je zapisana u fazi nosioca. Najjednostavnija inačica PSK modulacije je **BPSK** (*Binary PSK*) koja koristi samo dvije faze (0° i 180°), dok **QPSK** (*Quadrature PSK*) koristi četiri faze i time po simbolu prenosi dvostruko više bitova. **QAM** (*Quadrature Amplitude Modulation*) kombinira promjenu i amplitude i faze, što omogućuje još veći broj simbola pa time i veću spektralnu učinkovitost. Na primjer **16-QAM** koristi 16 različitih simbola i po simbolu prenosi 4 bita.

Napomena: ASK, FSK i PSK imaju nastavak *shift keying* jer se u njima radi

skok, odnosno pomak (engl. *shift*) kad se prelazi iz jednog stanja u drugo, a *keying* potječe iz stare telegrafije gdje je tipkalo (engl. key) služilo za uključivanje i isključivanje signala tijekom prijenosa digitalnih podataka.

Diskretna modulacija amplitude (ASK)

Kod diskretne modulacije amplitude (*Amplitude Shift Keying*, ASK) amplituda nosioca mijenja se ovisno o simbolu koji se prenosi. Modulirani signal može se zapisati kao

$$s(t) = A_i \cos(2\pi f_c t),$$

gdje je A_i amplituda pridružena simbolu i.

Za razliku od analogne amplitudne modulacije (AM), kod ASK modulacije amplituda ne može poprimiti proizvoljnu vrijednost, nego se bira iz unaprijed definiranog skupa amplituda. Svaka amplituda predstavlja jedno stanje moduliranog signala, odnosno jedan simbol. U najjednostavnijoj inačici koriste se samo dvije moguće amplitude. Tada govorimo o binarnoj ASK modulaciji (2-ASK). Jedna amplituda predstavlja bit 0, a druga bit 1.

Ako simbol sadrži više od jednog bita, koristi se više amplitudnih razina. Tada govorimo o M-ASK modulaciji gdje oznaka M predstavlja broj mogućih stanja. Na primjer, kod 4-ASK modulacije se koriste četiri različite amplitude koje mogu predstavljati četiri različita simbola.

Poseban slučaj ASK modulacije je **On-Off Keying (OOK)**. Kod OOK modulacije koristi se samo jedna nenulta amplituda:

$$s(t) = \begin{cases} A\cos(2\pi f_c t), & \text{ako je bit}=1 \\ 0, & \text{ako je bit}=0 \end{cases}$$

Drugim riječima, tijekom prijenosa bita 1 nosilac je prisutan, dok se tijekom prijenosa bita 0 nosilac ne odašilje. Zbog svoje jednostavnosti OOK je jedna od najjednostavnijih digitalnih modulacija te se koristi u nekim optičkim komunikacijama, infracrvenim daljinskim upravljačima i jednostavnim bežičnim sustavima.

Konstelacijski dijagram binarne ASK modulacije sastoji se od dvije točke smještene na osi I, budući da se mijenja samo amplituda signala, dok faza ostaje konstantna.

U nastavku je primjer ASK modulacije.

In [14]:

```
import numpy as np
import matplotlib.pyplot as plt

# poruka
bits = np.array([1, 0, 1, 1, 0, 0, 1, 0])

Tb = 1e-3                   # trajanje bita
fc = 5000                   # frekvencija nosioca
fs = 100 * fc               # frekvencija uzorkovanja

A0 = 0.3                    # amplituda za bit 0
A1 = 1.0                    # amplituda za bit 1

t_bit = np.arange(0, Tb, 1/fs)

signal = []
time = []

for i, bit in enumerate(bits):

    A = A1 if bit == 1 else A0

    s = A * np.cos(2*np.pi*fc*t_bit)

    signal.extend(s)
    time.extend(t_bit + i*Tb)

signal = np.array(signal)
time = np.array(time)

I = np.array([A0, A1])
Q = np.array([0, 0])

fig, ax = plt.subplots(3, 1, figsize=(10, 8))

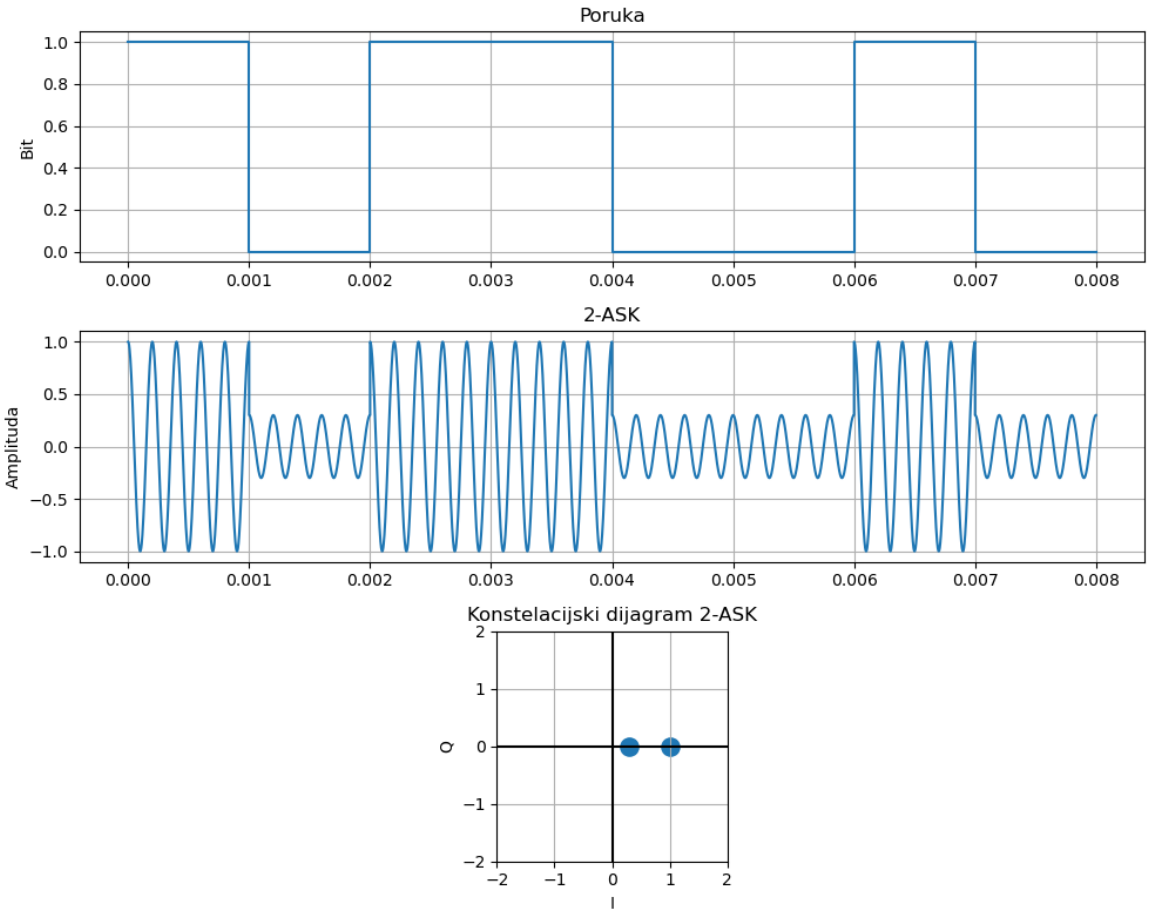
ax[0].step(np.arange(len(bits)+1)*Tb,
           np.append(bits, bits[-1]),
           where='post')
ax[0].set_ylabel('Bit')
ax[0].set_title('Poruka')
ax[0].grid()

ax[1].plot(time, signal)
ax[1].set_ylabel('Amplituda')
ax[1].set_title('2-ASK')
ax[1].grid()

ax[2].scatter(I, Q, s=120)
ax[2].axhline(0, color='k')
ax[2].axvline(0, color='k')
ax[2].set_xlabel('I')
ax[2].set_ylabel('Q')
ax[2].set_title('Konstelacijski dijagram 2-ASK')
ax[2].grid()
ax[2].set_aspect('equal')
ax[2].set_xticks([-2,-1, 0, 1, 2])
```

```
ax[2].set_yticks([-2,-1, 0, 1, 2])

plt.tight_layout()
plt.show()
```



Što vidimo?

U srednjem dijagramu prikazan je ASK signal u vremenskoj domeni. Može se primijetiti da frekvencija i faza nosioca ostaju nepromijenjene tijekom cijelog prijenosa. Jedina promjena događa se u amplitudi signala. Kada se promijeni vrijednost bita, amplituda nosioca prelazi na drugu unaprijed definiranu vrijednost.

U ovom konkretnom primjeru bitu 0 pridružena je amplituda $A_0 = 0.3$, dok je bitu 1 pridružena amplituda $A_1 = 1.0$. Na donjem dijagramu prikazan je konstelacijski dijagram ASK modulacije. Budući da se kod ASK-a mijenja samo amplituda, a faza ostaje konstantna, sve se točke nalaze na osi I. Komponenta Q jednaka je nuli za sve simbole.

Diskretna modulacija frekvencije (FSK)

Kod diskretne modulacije frekvencije (*Frequency Shift Keying*, FSK) frekvencija nosioca mijenja se ovisno o simbolu koji se prenosi. Modulirani

signal može se zapisati kao

$$s(t) = A\cos(2\pi f_i t),$$

gdje je A konstantna amplituda nosioca, a f_i frekvencija pridružena simbolu i.

Za razliku od analogne frekvencijske modulacije (FM), kod FSK modulacije frekvencija ne može poprimiti proizvoljnu vrijednost, nego se bira iz unaprijed definiranog skupa frekvencija. Svaka frekvencija predstavlja jedno stanje moduliranog signala, odnosno jedan simbol.

U najjednostavnijoj inačici koriste se samo dvije moguće frekvencije. Tada govorimo o binarnoj FSK modulaciji (2-FSK ili BFSK). Jedna frekvencija predstavlja bit 0, a druga bit 1. Ako simbol sadrži više od jednog bita, koristi se više frekvencijskih razina. Tada govorimo o M-FSK modulaciji gdje oznaka M predstavlja broj mogućih stanja. Na primjer, kod 4-FSK modulacije koriste se četiri različite frekvencije koje mogu predstavljati četiri različita simbola.

Budući da amplituda ostaje konstantna, informacija je sadržana isključivo u promjenama frekvencije. Zbog toga je FSK otporniji na amplitudni šum od ASK modulacije te se često koristi u radijskim komunikacijama, telemetriji i sustavima za prijenos podataka na većim udaljenostima.

Konstelacijski dijagram FSK modulacije razlikuje se od ASK i PSK modulacija jer simboli nisu određeni različitim amplitudama ili fazama, nego različitim frekvencijama. Zbog toga se FSK najčešće ne prikazuje konstelacijskim dijagramom, već promjenama frekvencije nosioca tijekom vremena.

U nastavku je primjer FSK modulacije.

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt

# poruka
bits = np.array([1, 0, 1, 1, 0, 0, 1, 0])

Tb = 1e-3                  # trajanje bita
fs = 500000                # frekvencija uzorkovanja

A = 1.0                    # amplituda

f0 = 3000                  # frekvencija za bit 0
f1 = 6000                  # frekvencija za bit 1

t_bit = np.arange(0, Tb, 1/fs)

signal = []
time = []

for i, bit in enumerate(bits):
```

```
    f = f1 if bit == 1 else f0

    s = A * np.cos(2*np.pi*f*t_bit)

    signal.extend(s)
    time.extend(t_bit + i*Tb)

signal = np.array(signal)
time = np.array(time)

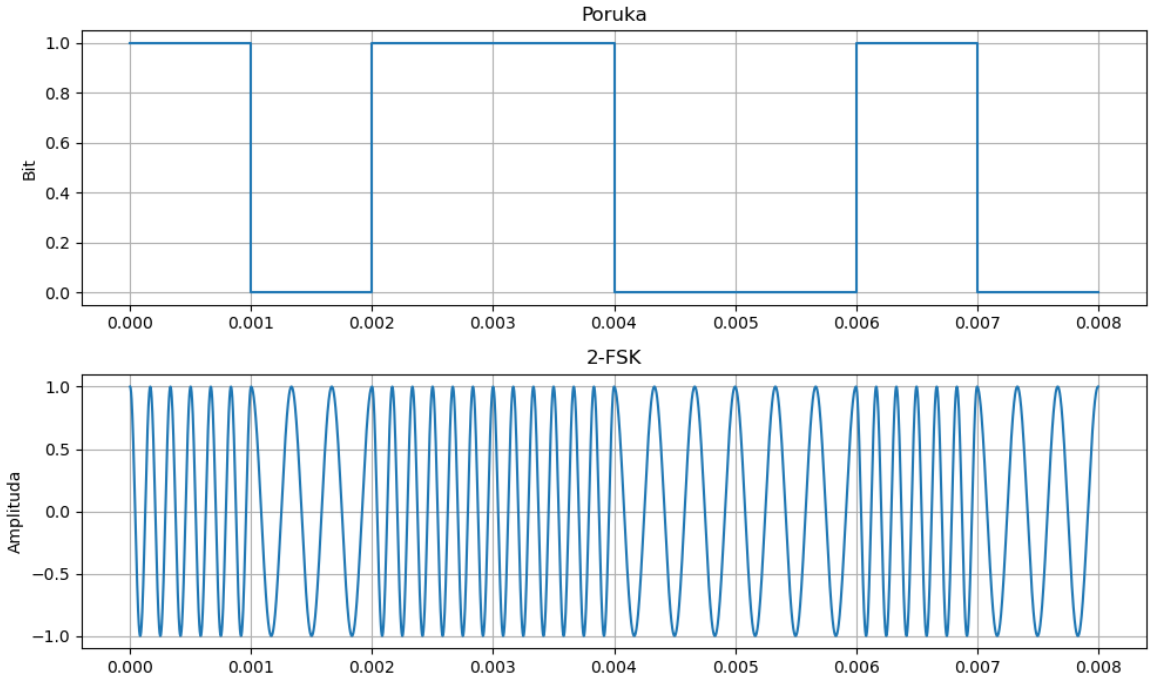
I = np.array([1, -1])
Q = np.array([0, 0])

fig, ax = plt.subplots(2, 1, figsize=(10, 6))

ax[0].step(np.arange(len(bits)+1)*Tb,
           np.append(bits, bits[-1]),
           where='post')
ax[0].set_ylabel('Bit')
ax[0].set_title('Poruka')
ax[0].grid()

ax[1].plot(time, signal)
ax[1].set_ylabel('Amplituda')
ax[1].set_title('2-FSK')
ax[1].grid()


plt.tight_layout()
plt.show()
```



Što vidimo?

U donjem dijagramu prikazan je FSK signal u vremenskoj domeni. Može se primijetiti da amplituda signala ostaje konstantna tijekom cijelog prijenosa.

Jedina promjena događa se u frekvenciji nosioca. Kada se promijeni vrijednost bita, signal prelazi s jedne unaprijed definirane frekvencije na drugu.

U ovom konkretnom primjeru bitu 0 pridružena je frekvencija $f_0 = 3\,\mathrm{kHz}$, dok je bitu 1 pridružena frekvencija $f_1 = 6\,\mathrm{kHz}$.

Diskretna modulacija faze (PSK)

Kod diskretne modulacije faze (*Phase Shift Keying*, PSK) faza nosioca mijenja se ovisno o simbolu koji se prenosi. Modulirani signal može se zapisati kao

$$s(t) = A\cos(2\pi f_c t + \varphi_i),$$

gdje je A amplituda nosioca, f_c frekvencija nosioca, a φ_i faza pridružena simbolu i.

Za razliku od analogne fazne modulacije (PM), kod PSK modulacije faza ne može poprimiti proizvoljnu vrijednost, nego se bira iz unaprijed definiranog skupa faznih pomaka. Svaki fazni pomak predstavlja jedno stanje moduliranog signala, odnosno jedan simbol.

Budući da amplituda i frekvencija ostaju konstantne, informacija je sadržana isključivo u promjenama faze nosioca. PSK modulacija se često koristi u suvremenim digitalnim komunikacijskim sustavima jer je manje osjetljiva na šum od ASK i zahtijeva manju snagu signala od FSK.

Konstelacijski dijagram PSK modulacije sastoji se od točaka raspoređenih na kružnici. Sve točke imaju jednaku udaljenost od ishodišta jer je amplituda konstantna, dok se razlikuju samo po kutu koji odgovara fazi pojedinog simbola.

Dvije su najčešće podvrste PSK modulacije: BPSK i QPSK.

Modulacija BPSK

Najjednostavniji oblik PSK modulacije je diskretna modulacija faze s dvama stanjima (engl. *Binary Phase Shift Keying*, BPSK). Kod BPSK modulacije koriste se dvije moguće faze koje se međusobno razlikuju za 180°.

Signal se može zapisati kao $s(t) = A\cos(2\pi f_c t + \varphi)$, pri čemu je

$$\varphi = \begin{cases} 0, & \text{za bit } 1, \\ \pi, & \text{za bit } 0. \end{cases}$$

Promjena faze za 180° znači da se signal za jedan bit može promatrati kao

pozitivna verzija nosioca, dok je za drugi bit signal invertiran.

Zbog velike udaljenosti između dvaju simbola, BPSK je vrlo otporna na šum te se koristi u satelitskim komunikacijama, telemetriji i drugim sustavima gdje je pouzdanost prijenosa važnija od brzine prijenosa podataka. Između ostalog, koristi se u letjelicama Voyager 1 i Voyager 2.

U nastavku je primjer BPSK modulacije u kojem je bitu 1 pridružena faza 0°, a bitu 0 je pridružena faza 180°. Promjena faze za 180° uzrokuje inverziju signala pa se sinusni val okreće oko vremenske osi.

Na donjem dijagramu prikazan je konstelacijski dijagram BPSK modulacije. Budući da postoje samo dva moguća simbola, dijagram sadrži dvije točke smještene na osi I. Jedna točka predstavlja fazu 0°, a druga fazu 180°. Komponenta Q jednaka je nuli za oba simbola.

In [4]:
```
import numpy as np
import matplotlib.pyplot as plt

# poruka
bits = np.array([1, 0, 1, 1, 0, 0, 1, 0])

Tb = 1e-3
fc = 5000
fs = 100 * fc

A = 1.0

t_bit = np.arange(0, Tb, 1/fs)

signal = []
time = []

for i, bit in enumerate(bits):

    phase = 0 if bit == 1 else np.pi

    s = A * np.cos(2*np.pi*fc*t_bit + phase)

    signal.extend(s)
    time.extend(t_bit + i*Tb)

signal = np.array(signal)
time = np.array(time)

I = np.array([1, -1])
Q = np.array([0, 0])

fig, ax = plt.subplots(3, 1, figsize=(10, 8))

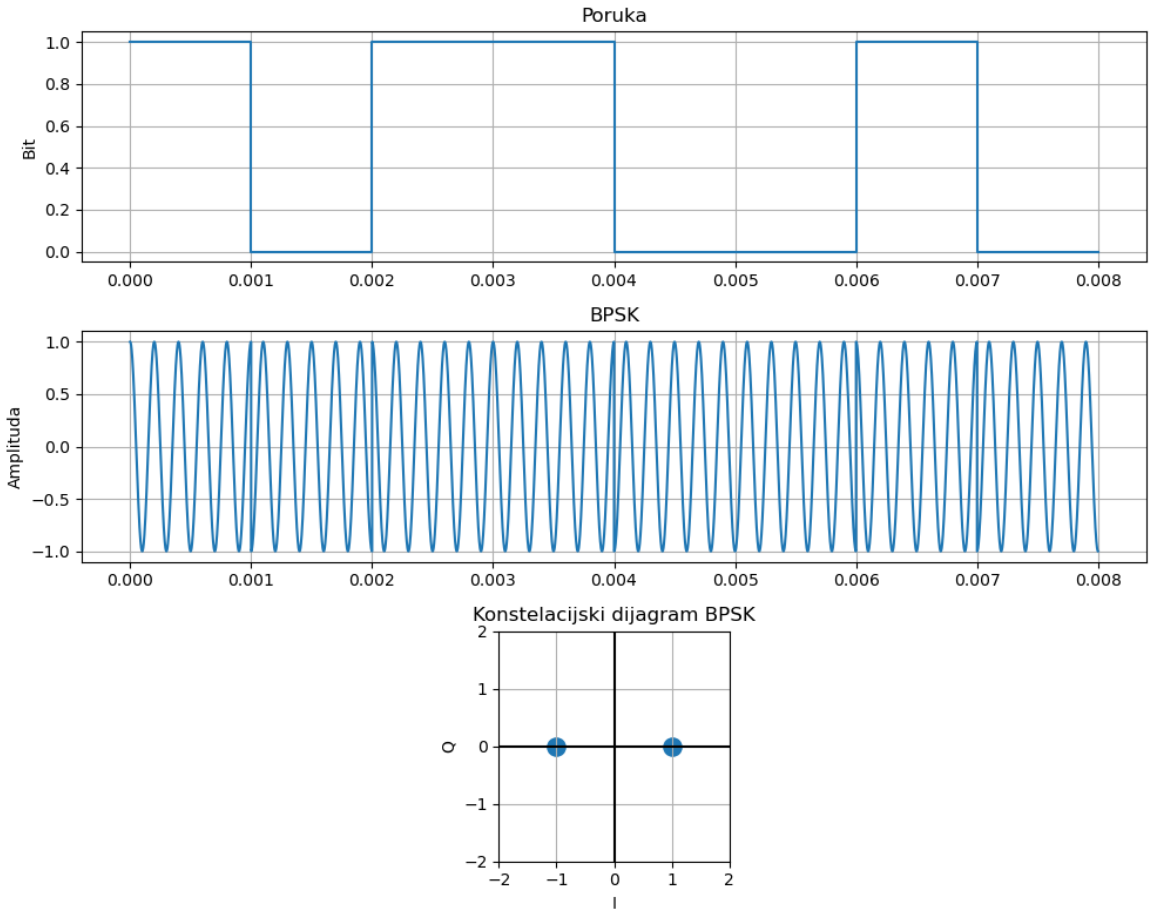
ax[0].step(np.arange(len(bits)+1)*Tb,
           np.append(bits, bits[-1]),
```

```
            where='post')
ax[0].set_ylabel('Bit')
ax[0].set_title('Poruka')
ax[0].grid()

ax[1].plot(time, signal)
ax[1].set_ylabel('Amplituda')
ax[1].set_title('BPSK')
ax[1].grid()

ax[2].scatter(I, Q, s=120)
ax[2].axhline(0, color='k')
ax[2].axvline(0, color='k')
ax[2].set_xlabel('I')
ax[2].set_ylabel('Q')
ax[2].set_title('Konstelacijski dijagram BPSK')
ax[2].grid()
ax[2].set_aspect('equal')
ax[2].set_xticks([-2,-1,0,1,2])
ax[2].set_yticks([-2,-1,0,1,2])

plt.tight_layout()
plt.show()
```



Modulacija QPSK

Diskretna modulacija faze s četirima stanjima (engl. *Quadrature Phase Shift*

Keying, QPSK) proširenje je BPSK modulacije u kojem se koriste četiri različita fazna stanja. Budući da postoje četiri moguća simbola, svaki simbol može predstavljati dva bita informacije.

Signal se može zapisati kao $s(t) = A\cos(2\pi f_c t + \varphi_i)$, gdje faza φ_i poprima jednu od četiri moguće vrijednosti.

Najčešće korištene faze su $45^\circ,\ 135^\circ,\ 225^\circ,\ 315^\circ$.

Kako bi se smanjio broj bitnih pogrešaka uzrokovanih šumom, simboli se najčešće raspoređuju prema **Grayevom kodiranju** (engl. *Gray coding*). Kod Grayevog kodiranja susjedni simboli razlikuju se u samo jednom bitu.

Jedan od mogućih rasporeda simbola je

Bitovi	Faza
00	45°
01	135°
11	225°
10	315°

Na taj način pogreška pri odlučivanju o susjednom simbolu najčešće uzrokuje pogrešku samo jednog bita.

QPSK prenosi dva bita po simbolu, što znači da pri istoj brzini simbola može ostvariti dvostruko veću brzinu prijenosa podataka od BPSK modulacije. Zbog toga se vrlo često koristi u bežičnim komunikacijama, satelitskim vezama, Wi-Fi sustavima i mobilnim mrežama.

Konstelacijski dijagram QPSK modulacije sastoji se od četiri jednako razmaknute točke na kružnici. Sve točke imaju jednaku amplitudu, dok se razlikuju po fazi. Razmak između susjednih točaka iznosi 90°.

U nastavku je primjer za QPSK modulaciju. Srednji dijagram prikazuje QPSK signal u vremenskoj domeni. Amplituda i frekvencija nosioca ostaju konstantne, a faza se mijenja između četiri moguće vrijednosti. Svaka faza predstavlja jedan simbol koji sadrži dva bita informacije. U ovom primjeru korišteno je Grayevo kodiranje. Susjedni simboli u konstelacijskom dijagramu razlikuju se samo u jednom bitu, čime se smanjuje broj bitnih pogrešaka u slučaju pogrešne odluke prijamnika.

In [5]:
```
import numpy as np
import matplotlib.pyplot as plt

# bitovi (paran broj)
```

```
bits = np.array([0,0, 0,1, 1,1, 1,0])

Tb = 1e-3
Ts = 2 * Tb               # trajanje simbola

fc = 5000
fs = 100 * fc
A = 1.0

t_sym = np.arange(0, Ts, 1/fs)

# Gray kodiranje
mapping = {
    (0,0): np.pi/4,
    (0,1): 3*np.pi/4,
    (1,1): 5*np.pi/4,
    (1,0): 7*np.pi/4
}

signal = []
time = []

for k in range(0, len(bits), 2):

    symbol = (bits[k], bits[k+1])

    phase = mapping[symbol]

    s = A * np.cos(2*np.pi*fc*t_sym + phase)

    signal.extend(s)
    time.extend(t_sym + (k//2)*Ts)

signal = np.array(signal)
time = np.array(time)

# konstelacija
I = np.cos(np.array([
    np.pi/4,
    3*np.pi/4,
    5*np.pi/4,
    7*np.pi/4
]))

Q = np.sin(np.array([
    np.pi/4,
    3*np.pi/4,
    5*np.pi/4,
    7*np.pi/4
]))

labels = ['00', '01', '11', '10']

fig, ax = plt.subplots(3, 1, figsize=(10, 8))

ax[0].step(np.arange(len(bits)+1)*Tb,
```

```
            np.append(bits, bits[-1]),
            where='post')
ax[0].set_ylabel('Bit')
ax[0].set_title('Poruka')
ax[0].grid()

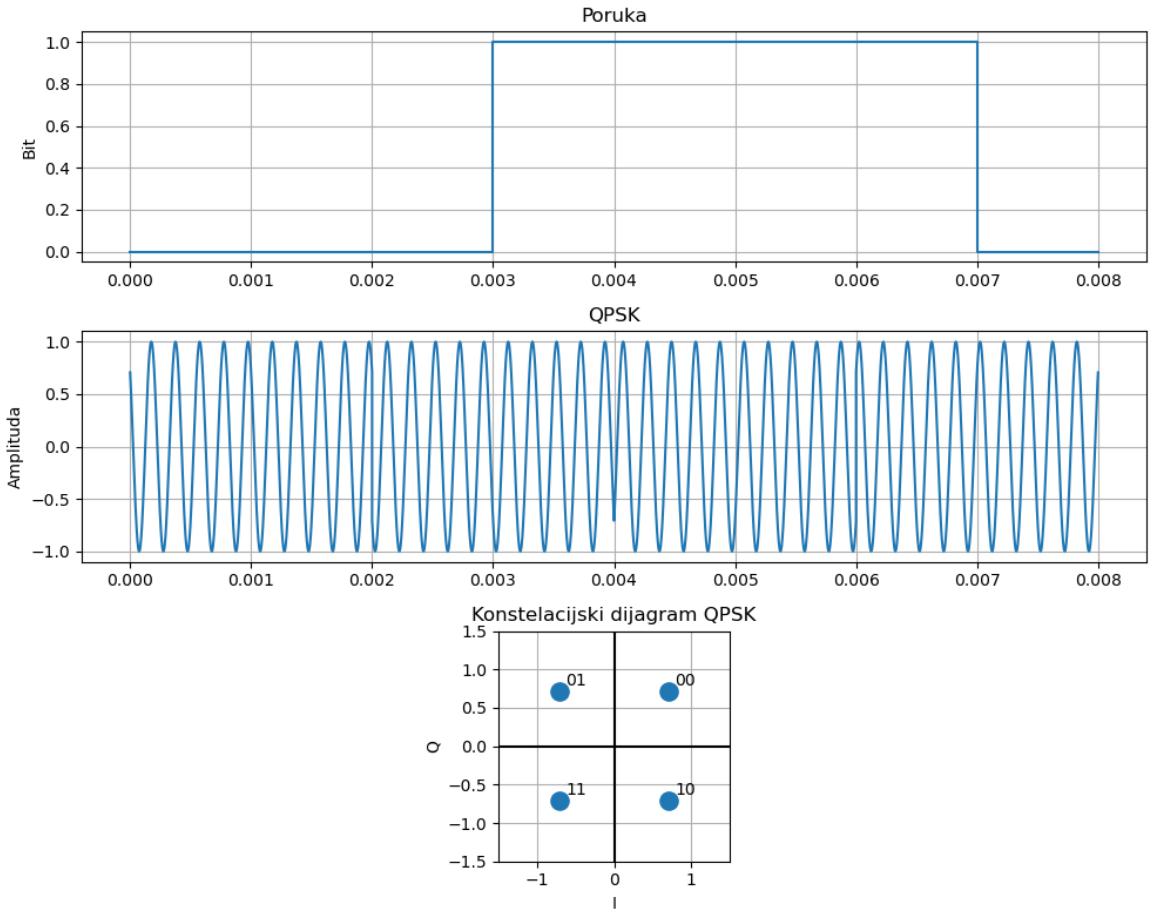
ax[1].plot(time, signal)
ax[1].set_ylabel('Amplituda')
ax[1].set_title('QPSK')
ax[1].grid()

ax[2].scatter(I, Q, s=120)

for x, y, label in zip(I, Q, labels):
    ax[2].text(x+0.08, y+0.08, label)

ax[2].axhline(0, color='k')
ax[2].axvline(0, color='k')
ax[2].set_xlabel('I')
ax[2].set_ylabel('Q')
ax[2].set_title('Konstelacijski dijagram QPSK')
ax[2].grid()
ax[2].set_aspect('equal')
ax[2].set_xlim(-1.5, 1.5)
ax[2].set_ylim(-1.5, 1.5)

plt.tight_layout()
plt.show()
```



Kvadraturna amplitudna modulacija (QAM)

Kod kvadraturne amplitudne modulacije (engl. *Quadrature Amplitude Modulation*, QAM) informacija se prenosi promjenom i amplitude i faze nosioca pa se QAM može promatrati kao kombinacija ASK i PSK modulacije.

Modulirani signal pišemo

$$s(t) = I\cos(2\pi f_c t) - Q\sin(2\pi f_c t),$$

gdje su I (*In-phase*) i Q (*Quadrature*) komponente signala.

Svaki simbol određen je parom vrijednosti (I, Q), pa se simboli prirodno prikazuju u konstelacijskom dijagramu. Položaj točke u dijagramu određuje amplitudu i fazu odgovarajućeg signala.

Za M mogućih simbola radimo M-QAM modulaciju. Broj bitova po simbolu tada je jednak $\log_2 M$.

Na primjer:

- 16-QAM prenosi 4 bita po simbolu,
- 64-QAM prenosi 6 bita po simbolu,
- 256-QAM prenosi 8 bita po simbolu.

Kod 16-QAM modulacije svaki simbol sadrži četiri bita informacije. Uobičajeno se prva dva bita koriste za određivanje razine komponente I, dok druga dva bita određuju razinu komponente Q. U praksi se vrlo često koristi Grayevo kodiranje. Susjedni simboli u konstelacijskom dijagramu razlikuju se samo u jednom bitu, čime se smanjuje broj bitnih pogrešaka uzrokovanih šumom.

Prednost QAM modulacije je vrlo velika spektralna učinkovitost jer jedan simbol može prenositi velik broj bitova. Zbog toga se koristi u Wi-Fi mrežama, digitalnoj televiziji, mobilnim mrežama i mnogim drugim suvremenim komunikacijskim sustavima. S druge strane, nedostatak je veća osjetljivost na šum i izobličenja. Kako broj simbola raste, točke u konstelacijskom dijagramu postaju sve bliže jedna drugoj pa prijamnik teže razlikuje pojedine simbole. Zbog toga više razine modulacije, poput 64-QAM ili 256-QAM, zahtijevaju kvalitetniji komunikacijski kanal i veći omjer signal-šum.

U nastavku je primjer 16-QAM modulacije.

In [6]:
```
import numpy as np
import matplotlib.pyplot as plt

# poruka
bits = np.array([
```

```
    0,0,0,0,
    0,0,0,1,
    0,0,1,1,
    0,0,1,0
])

Tb = 1e-3
Ts = 4 * Tb

fc = 5000
fs = 100 * fc

t_sym = np.arange(0, Ts, 1/fs)

# Gray kodirana mapa
mapping = {
    (0,0,0,0): (-3,  3),
    (0,0,0,1): (-1,  3),
    (0,0,1,1): ( 1,  3),
    (0,0,1,0): ( 3,  3),

    (0,1,0,0): (-3,  1),
    (0,1,0,1): (-1,  1),
    (0,1,1,1): ( 1,  1),
    (0,1,1,0): ( 3,  1),

    (1,1,0,0): (-3, -1),
    (1,1,0,1): (-1, -1),
    (1,1,1,1): ( 1, -1),
    (1,1,1,0): ( 3, -1),

    (1,0,0,0): (-3, -3),
    (1,0,0,1): (-1, -3),
    (1,0,1,1): ( 1, -3),
    (1,0,1,0): ( 3, -3)
}

signal = []
time = []

for k in range(0, len(bits), 4):

    symbol = tuple(bits[k:k+4])

    I, Q = mapping[symbol]

    s = I*np.cos(2*np.pi*fc*t_sym) - \
        Q*np.sin(2*np.pi*fc*t_sym)

    signal.extend(s)
    time.extend(t_sym + (k//4)*Ts)

signal = np.array(signal)
time = np.array(time)

# konstelacija
```

```
levels = [-3, -1, 1, 3]
I_const = []
Q_const = []

for i in levels:
    for q in levels:
        I_const.append(i)
        Q_const.append(q)

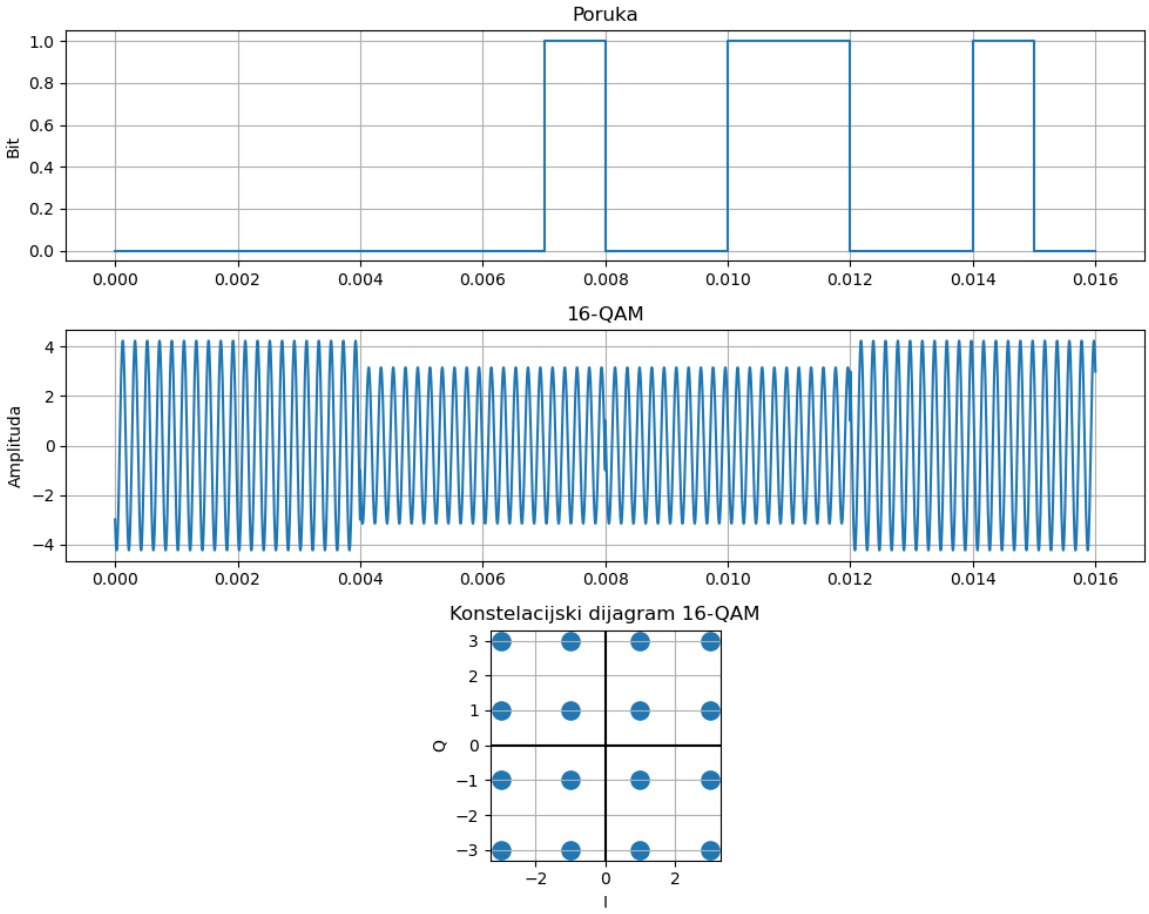
fig, ax = plt.subplots(3, 1, figsize=(10, 8))

ax[0].step(np.arange(len(bits)+1)*Tb,
           np.append(bits, bits[-1]),
           where='post')
ax[0].set_ylabel('Bit')
ax[0].set_title('Poruka')
ax[0].grid()

ax[1].plot(time, signal)
ax[1].set_ylabel('Amplituda')
ax[1].set_title('16-QAM')
ax[1].grid()

ax[2].scatter(I_const, Q_const, s=120)
ax[2].axhline(0, color='k')
ax[2].axvline(0, color='k')
ax[2].set_xlabel('I')
ax[2].set_ylabel('Q')
ax[2].set_title('Konstelacijski dijagram 16-QAM')
ax[2].grid()
ax[2].set_aspect('equal')

plt.tight_layout()
plt.show()
```



Što vidimo?

U srednjem dijagramu prikazan je 16-QAM signal u vremenskoj domeni. Za razliku od ASK, FSK i PSK modulacija, ovdje se istodobno mijenjaju amplituda i faza signala. Zbog toga signal može poprimiti veći broj različitih stanja.

U ovom primjeru svaki simbol predstavlja četiri bita informacije. Svaka kombinacija četiriju bitova preslikava se u jednu točku konstelacijskog dijagrama određenu koordinatama (I, Q).

Na kraju je prikazan konstelacijski dijagram 16-QAM modulacije. Simboli su raspoređeni u mreži od 16 točaka. Horizontalni položaj određuje vrijednost komponente I, dok vertikalni položaj određuje vrijednost komponente Q. Za razliku od PSK modulacija (npr. BPSK i QPSK), gdje sve točke leže na kružnici, kod QAM-a se mijenjaju i amplituda i faza pa se simboli nalaze na različitim udaljenostima od ishodišta.

Budući da postoji 16 mogućih simbola, svaki simbol prenosi $\log_2(16) = 4$ bita informacije. Upravo zbog mogućnosti prijenosa više bitova po simbolu QAM modulacije postižu veće brzine prijenosa od ASK, FSK i PSK modulacija.

Brzina prijenosa simbola i bita

Brzina prijenosa simbola označava koliko se simbola prenese u jednoj sekundi i označava se s R_s. Jedinica je simbol u sekundi, odnosno (po francuskom inženjeru Émileu Baudotu) **baud** (Bd).

$$R_s = \frac{1}{T_s},$$

gdje je T_s trajanje jednog simbola.

Brzina prijenosa bita označava koliko se bitova prenese u jednoj sekundi i označava se s R_b. Jedinica je bit u sekundi (*bit/s*). Ako svaki simbol sadrži k bitova informacije, tada vrijedi

$$R_b = k \cdot R_s,$$

Ako je brzina simbola

$$R_s = 1000\ \text{baud},$$

tada su odgovarajuće brzine prijenosa bita:

Modulacija	Bitova po simbolu (k)	Brzina prijenosa bita (R_b)
BPSK	1	1000 bit/s
QPSK	2	2000 bit/s
16-QAM	4	4000 bit/s
64-QAM	6	6000 bit/s

Iz tablice se vidi da povećanjem broja mogućih simbola možemo prenijeti više bitova tijekom jednog trajanja simbola. Zbog toga modulacije poput QPSK, 16-QAM i 64-QAM omogućuju veće brzine prijenosa podataka bez povećanja brzine simbola. Pritom treba imati na umu da veći broj simbola znači i manji razmak između točaka u konstelacijskom dijagramu. Zato su modulacije s velikim brojem stanja osjetljivije na šum i zahtijevaju kvalitetniji komunikacijski kanal.

IQ modulator

Kod suvremenih komunikacijskih sustava digitalna se modulacija najprije izvodi u osnovnom pojasu (engl. *baseband*), pa se nakon toga signal podiže na željenu radijsku frekvenciju. Takav pristup omogućuje jednostavniju obradu signala i korištenje istog hardvera za različite modulacijske postupke.

U slučaju digitalne komunikacije ulaz predstavlja niz bitova koje želimo

prenijeti komunikacijskim kanalom. Ti se bitovi najprije obrađuju u **baseband procesoru**. Njegova je zadaća odabranu digitalnu modulaciju primijeniti na ulazni niz podataka i generirati I i Q komponente.

I i Q komponente se mogu prenijeti zasebno pomoću dva međusobno ortogonalna nositelja. U tu se svrhu koriste sinus i kosinus iste frekvencije. Budući da su sinus i kosinus međusobno ortogonalni, informacije sadržane u I i Q komponenti ne ometaju jedna drugu te se mogu istovremeno prenositi istim radijskim signalom.

Kako bi se I i Q komponente pretvorile u visokofrekvencijski signal prikladan za odašiljanje, koristi se **IQ modulator**. Njegov je zadatak jednostavan: I komponenta množi se s kosinusnim nositeljem ($\cos(2\pi f_c t)$), Q komponenta množi se sa sinusnim nositeljem ($\sin(2\pi f_c t)$). Dobiveni se signali zatim zbrajaju te nastaje visokofrekvencijski signal:

$$s(t) = I(t)\cos(2\pi f_c t) - Q(t)\sin(2\pi f_c t).$$

Važno je uočiti da IQ modulator ne određuje vrstu modulacije. On ne zna jesu li I i Q komponente nastale BPSK, QPSK ili QAM modulacijom. Sve informacije o odabranoj modulaciji već su sadržane u I i Q komponentama koje dolaze iz baseband procesora. IQ modulator uvijek obavlja isti postupak: množi I komponentu s kosinusom, Q komponentu sa sinusom te zbraja dobivene rezultate. Dakle, baseband procesor zadužen je za obradu podataka i generiranje I i Q komponenata, dok je IQ modulator zadužen za njihovu pretvorbu u visokofrekvencijski signal. Budući da su te dvije funkcije odvojene, ista se hardverska implementacija IQ modulatora može koristiti za velik broj različitih digitalnih modulacija.

Ta je ideja posebno važna kod softverski definiranog radija (engl. *Software Defined Radio*, **SDR**). SDR je radijski uređaj kod kojeg se velik dio obrade signala ostvaruje softverom umjesto namjenskim hardverom. Takav uređaj sadrži RF sklopove potrebne za generiranje i prijam radijskih signala: lokalne oscilatore, filtre, pojačala, analogno-digitalne i digitalno-analogne pretvornike te **IQ modulatore i demodulatore**. Za razliku od klasičnih komunikacijskih uređaja, kod SDR-a se mnoge funkcije mogu mijenjati jednostavnom promjenom softvera bez potrebe za izmjenom hardvera.

U laboratorijskim vježbama koristili smo ADALM-Pluto SDR. U toj konfiguraciji računalo preuzima ulogu baseband procesora. Programiranjem smo generirali niz bitova, odabrali modulaciju i izračunali I i Q uzorke. Ti se uzorci zatim šalju prema Pluto SDR uređaju. Pluto SDR koristi svoje RF sklopove i ugrađeni IQ modulator kako bi iz primljenih I i Q uzoraka generirao visokofrekvencijski signal koji se može odašiljati.

Implementacija za SDR

Za komunikaciju s Pluto SDR uređajem iz Pythona koristili smo biblioteku **`pyadi-iio`** koja omogućuje izravno upravljanje i konfiguraciju uređaja te slanje i primanje IQ uzoraka. Osnovna upotreba svodi se na tri koraka: inicijalizaciju uređaja, postavljanje parametara te slanje ili primanje uzoraka.

U nastavku je primjer programskog koda za ADALM-Pluto SDR.

In [ ]:
```
import adi
import numpy as np

# Inicijalizacija uređaja
sdr = adi.Pluto("ip:192.168.2.1")

# Konfiguracija
sdr.sample_rate = 1_000_000       # Frekvencija uzorkovanja [Hz]
sdr.tx_lo = 915_000_000           # Frekvencija nosioca za predajni
sdr.rx_lo = 915_000_000           # Frekvencija nosioca za prijamni
sdr.tx_hardwaregain_chan0 = -20   # TX pojačanje [dB]
sdr.rx_hardwaregain_chan0 = 30    # RX pojačanje [dB]
sdr.rx_buffer_size = 1024         # Broj uzoraka po pozivu rx()

# Generiranje I/Q uzoraka
N = 1024
t = np.arange(N)
iq_tx = np.exp(1j * 2 * np.pi * 0.1 * t).astype(np.complex64) * 2**

# Slanje uzoraka
sdr.tx_cyclic_buffer = True
sdr.tx(iq_tx)

# Primanje uzoraka
iq_rx = sdr.rx()
```